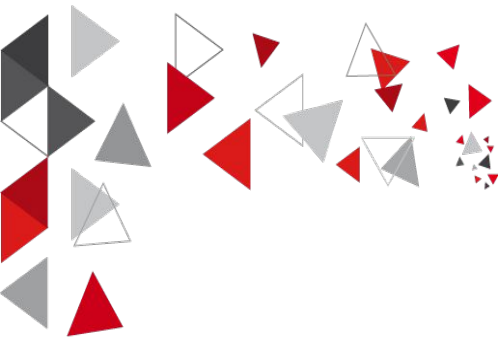
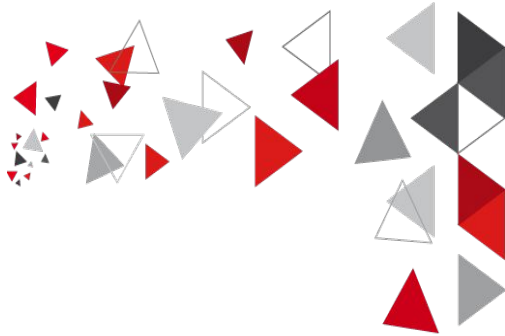


esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Conception Orienté Objet et Programmation Java

Chapitre 11 : Interfaces fonctionnelles & Expressions Lambda

Objectifs du chapitre



- Découvrir les Interfaces fonctionnelles
- Instancier des interfaces avec un objet anonyme
- Instancier des interfaces fonctionnelles avec l'expression lambda
- Découvrir les méthodes de référence

Interface fonctionnelle



Les interfaces fonctionnelles



Une **interface fonctionnelle** est une interface Java qui ne comporte qu'**une seule méthode abstraite**, permettant de représenter un comportement ou une fonction spécifique.

Elle peut également contenir des méthodes par **défaut ou statiques**, mais elle doit avoir **une et une seule** méthode abstraite.

Une interface fonctionnelle pourra être annotée avec l'annotation

@FunctionalInterface

L'annotation **@FunctionalInterface** n'est pas obligatoire, elle a un rôle important permettant au compilateur de forcer l'interface afin qu'elle ne contienne qu'une seule méthode abstraite

```
@FunctionalInterface   //optionnel
interface Math1{

    int add(int x,int y);

    default int divide(int x,int y){
        return x / y;
    }
}
```

Si nous essayons de définir plus d'une seule méthode abstraite dans une interface annotée avec **@FunctionalInterface**, le compilateur affichera une erreur.

```
@FunctionalInterface   //erreur de compilation
interface Math1{

    int add(int x,int y);
    int multiply(int x,int y);

    default int divide(int x,int y){
        return x / y;
    }
}
```

Les interfaces fonctionnelles: instanciation



Une interface fonctionnelle peut être instanciée à condition que vous fournissiez une implémentation de sa méthode abstraite.

Elles offrent une flexibilité accrue pour personnaliser les comportements en fournissant des implémentations de méthodes, **éliminant** ainsi la nécessité de créer **plusieurs classes** qui implémentent l'interface.

Les interfaces fonctionnelles: instanciation

```
public interface Math {
    int calculate(int x, int y);
}
```

```
//méthode main
Math add = new Math() {
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }
};

int sum = add.calculate(7, 8); //sum = 15

Math multiply = new Math() {
    @Override
    public int calculate(int x, int y) {
        return x * y;
    }
};

int result = multiply.calculate(7, 8); //result = 56
```

Objet anonyme

Les interfaces fonctionnelles: instanciation



Un **objet anonyme** est une instance d'une interface/classe abstraite sans nom. Les objets anonymes sont généralement utilisés dans des endroits où une classe complète n'est pas nécessaire et où la création d'une classe dédiée peut sembler lourde.

L'instanciation d'une interface via un **objet anonyme** signifie qu'on peut créer plusieurs implémentations de sa méthode **abstraite** sans avoir à créer une classe dédiée pour chaque scénario spécifique. Cela simplifie considérablement le code en évitant la surcharge de classes.

Lambda Expression

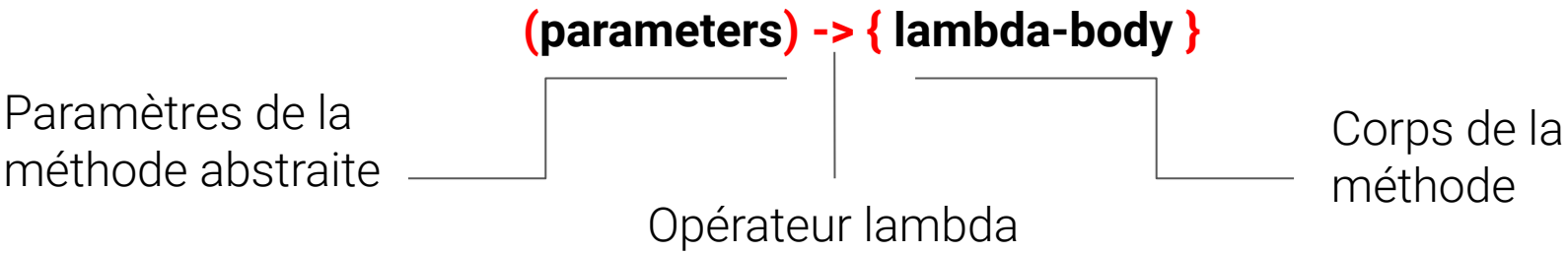


Lambda Expression est une manière simplifiée pour l'implémentation d'une interface qui n'a qu'une seule méthode abstraite (SAM).

-> SAM (Single Abstract Method) Interface = interface fonctionnelle

On Peut simplifier un objet anonyme d'une interface fonctionnelle en l'écrivant sous format d'une Lambda Expression.

L'expression Lambda s'écrit d'une façon assez simple en suivant quelques règles:



Expression Lambda : Comment implémenter ?

Pour appliquer l'expression lambda, nous commencerons par la création d'un objet anonyme.

```
public interface Math {
    int calculate(int x, int y);
}
```

```
Math add = new Math() {
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }
};

int sum = add.calculate(7, 8);
```

Expression Lambda : Comment implémenter ?

Le compilateur identifie l'interface à instancier à travers son type, donc il n'est pas nécessaire de spécifier son nom.

=> On supprime donc la partie d'instanciation.

```
Math add = new Math() {
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }
};

int sum = add.calculate(7, 8);
```



```
Math add =
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }


int sum = add.calculate(7, 8);
```

Expression Lambda : Comment implémenter ?

Le compilateur reconnaît qu'il s'agit d'une interface avec une seule méthode abstraite, ainsi que sa visibilité et son type de retour.

=> On supprime donc le nom de la méthode, son modificateur et le type de retour.

```
Math add =
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }


int sum = add.calculate(7, 8);
```



```
Math add = (int x, int y) {
        return x + y;
    }

int sum = add.calculate(7, 8);
```

Expression Lambda : Comment implémenter ?

Maintenant il suffit d'ajouter l'opérateur lambda (->).

```
Math add = new Math() {
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }
};

int sum = add.calculate(7, 8);
```



```
Math add = (int x, int y) -> {
        return x + y;
    };

int sum = add.calculate(7, 8);
```

Expression Lambda : Comment implémenter ?



On peut encore simplifier cette expression en appliquant ces règles:

- Si le corps de l'expression se compose d'une seule instruction :
  - Aucun besoin d'accolades.
  - Aucun besoin d'utiliser le mot clé "return".
- Les lambdas avec un seul paramètre n'ont pas besoin de parenthèses.
- Les lambdas sans paramètres doivent inclure des parenthèses.
- Les lambdas avec plus d'un paramètre nécessitent l'utilisation de parenthèses.
- La spécification des types de paramètres est optionnelle

Expression Lambda : Comment implémenter ?

L'utilisation de lambdas au lieu d'objets anonymes permet donc de réduire la quantité de code nécessaire grâce à une syntaxe concise.

Résultat final:

```
Math add = new Math() {
    @Override
    public int calculate(int x, int y) {
        return x + y;
    }
};

int sum = add.calculate(7, 8);
```



```
Math add = (x, y) -> x + y;

int sum = add.calculate(7, 8);
```

Les interfaces fonctionnelles prédéfinies



Les interfaces fonctionnelles prédéfinies

Interface fonctionnelle	Description	Argument(s)	Type de retour	Méthode abstraite
Comparator<T>	compare deux objets de type T	T, T	int	compare(T t1, T t2)
Supplier<T>	Fournit une valeur de type T.	Aucun	T	get()
Predicate<T>	Vérifie si une valeur de type T satisfait une condition.	T	boolean	test(T t)
Consumer<T>	Consomme une valeur de type T sans retour.	T	void	accept(T t)
Function<T, R>	Transforme une valeur de type T en une valeur de type R.	T	R	apply(T t)

Les interfaces fonctionnelles prédéfinies

Interface fonctionnelle	Description	Argument(s)	Type de retour	Méthode abstraite
BiFunction<T, U, R>	Transforme deux valeurs de type T et U en une valeur de type R.	T, U	R	apply(T t, U u)
BiPredicate<T, U>	Vérifie si deux valeurs de type T et U satisfont une condition.	T, U	boolean	test(T t, U u)
BiConsumer<T, U>	Consomme deux valeurs de type T et U sans retour.	T, U	void	accept(T t, U u)
UnaryOperator<T>	Transforme une valeur de type T en une valeur de type T.	T	T	apply(T t)
BinaryOperator<T>	Combine deux valeurs de type T en une seule valeur de type T.	T, T	T	apply(T t1, T t2)

Interfaces fonctionnelles & Expression Lambda : Appliquons (1/8)



Function: Une interface fonctionnelle représentant une méthode qui prend un type d'entrée et renvoie un type de sortie.

```
public interface Function<T, R> {

    R apply(T t);
}
```

```
// Définition d'un "Function" qui prend un String et retourne sa longueur
Function<String, Integer> fun = s -> s.length();

// Application de la fonction à la chaîne "Hello" pour obtenir la longueur
int x = fun.apply("Hello"); //x = 5
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (2/8)



Consumer: Une interface fonctionnelle représentant une action qui prend un type d'entrée mais ne renvoie rien (void).

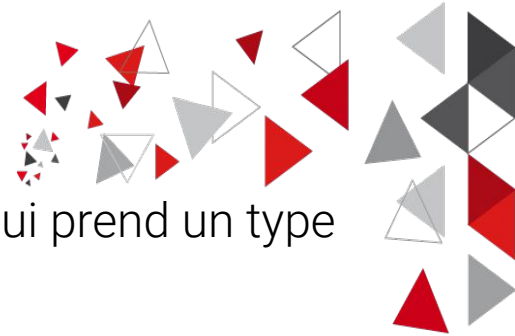
```
public interface Consumer<T> {

    void accept(T t);
}
```

```
// Définition d'un "Consumer" qui prend un String et effectue une action (affichage)
Consumer<String> con = s -> System.out.println("Hello " + s);

// Application du "Consumer" à la chaîne "3A" pour effectuer l'action (affichage)
con.accept("3A"); //Output: Hello 3A
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (3/8)



Predicate: Une interface fonctionnelle représentant un test, qui prend un type d'entrée et renvoie un booléen.

```
public interface Predicate<T> {

    boolean test(T t);
}
```

```
// Définition d'un "Predicate" qui prend un String et vérifie une condition
Predicate<String> pre = s -> s.startsWith("n");

// Application du "Predicate" avec "Hamdi" pour vérifier si elle commence par "n"
boolean check = pre.test("Hamdi"); //check = false
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (4/8)



Supplier: Une interface fonctionnelle représentant une fourniture qui ne prend aucun argument et renvoie un type spécifié.

```
public interface Supplier<T> {

    T get();
}
```

```
// Définition d'un "Supplier" qui renvoie une nouvelle instance de la classe String initialisée avec "Hello"
Supplier<String> sup = () -> new String("Hello");

// Application du "Supplier" pour récupérer la nouvelle chaîne créée
String txt = sup.get(); // txt = "Hello"
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (5/8)



Comparator: Une interface fonctionnelle représentant un comparateur qui compare deux objets pour déterminer leur ordre.

```
public interface Comparator<T> {

    int compare(T o1, T o2);
}
```

```
// Définition d'une "Comparator" qui compare deux chaînes
Comparator<String> comp = (a, b) -> a.compareTo(b);

// Application du "Comparator" pour comparer les chaînes "Ala" et "Ali"
int result  = comp.compare("Ala", "Ali"); // result = -8 => "Ala" < "Ali"
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (6/8)

BiFunction: Une interface fonctionnelle représentant une opération prenant deux arguments de types différents et renvoyant un résultat.

```
public interface BiFunction<T, U, R> {

    R apply(T t, U u);
}
```

```
// Définition d'un "BiFunction" qui prend une chaîne (a) et un entier (b),
multiplie la longueur de la chaîne par l'entier, puis renvoie le résultat sous
forme de Float
BiFunction<String, Integer, Float> bif = (a, b) -> Float.valueOf(a.length() * b);

// Application du "BiFunction " avec la chaîne "Hello" et l'entier 3
float x = bif.apply("Hello", 3); // x = 15.0
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (7/8)



BinaryOperator: Une interface fonctionnelle similaire à BiFunction, mais opérant sur deux opérandes du même type, et renvoyant un résultat du même type que les opérandes.

```
public interface BinaryOperator<T> extends BiFunction<T,T,T> {

    T apply(T t, T u);
}
```

```
// Définition d'une opération sur deux opérandes du même type (ici, multiplication)
BinaryOperator<Integer> bin = (a, b) -> a * b;

// Application du "UnaryOperator" avec l'entier 2
int x = bin.apply(2); // x = 4
```

Interfaces fonctionnelles & Expression Lambda : Appliquons (8/8)



UnaryOperator: Une interface fonctionnelle représentant une opération sur un seul opérande, prenant et renvoyant le même type.

```
public interface UnaryOperator<T> extends Function<T,T> {

    T apply(T t);
}
```

```
// Définition d'une "UnaryOperator" qui prend un entier et renvoie le carré de cet entier
UnaryOperator<Integer> un = a -> a * a;

// Application du "BinaryOperator" avec les entiers 2 et 5
int x = un.apply(2); // x = 10
```

Les méthodes de référence



Les méthodes de référence

Les **méthodes de référence** sont un type particulier d'expressions lambda. Ils sont souvent utilisés pour créer des expressions lambda simples en faisant référence à des méthodes existantes. Leur utilisation est généralement associé avec **“Stream”**.

Il existe quatre types de références de méthodes :

- Référence à une méthode statique.
- Référence à une méthode d'instance d'un objet particulier.
- Référence à une méthode d'instance d'un type particulier.
- Référence à un constructeur.

Référence à une méthode statique

Une méthode de référence vers une méthode statique permet d'appeler une méthode statique existante en utilisant l'opérateur de référence (**::**)

Syntaxe : NomDeLaClasse::nomDeLaMethodeStatique

Exemple:

Si une classe existe déjà pour calculer le carré d'un entier, il n'est pas nécessaire de créer une expression lambda pour ce cas, car la méthode correspondante est déjà disponible.

Référence à une méthode statique

```
class MathUtils {
   // Méthode statique qui calcule le carré d'un entier
   public static int carre(int x) {
       return x * x;
   }
}
```

On remarque que cette méthode ressemble au comportement d'un **UnaryOperator** car elle prend un entier en entrée et renvoie un entier. Dans ce cas, il est possible d'utiliser l'opérateur de référence (**::**) pour appeler la méthode **"carre"** plutôt que de fournir une expression lambda.

```
// Utilisation de la référence à la méthode statique
UnaryOperator<Integer> carre = MathUtils::carre;
int y = carre.apply(12);
```

Référence à une méthode d'instance d'un objet particulier



Une méthode de référence vers une méthode d'instance d'un objet permet d'appeler une méthode existante d'un objet déjà créé en utilisant l'opérateur de référence (**::**)

Syntaxe : nomObjet::nomDeLaMethode

Exemple:

Si une classe existe déjà pour retourner le nombres de caractères dans une chaîne, il n'est pas nécessaire de créer une expression lambda pour ce cas, car la méthode correspondante est déjà disponible.

Référence à une méthode d'instance d'un objet particulier

```
class StringUtils {
   public int getNumberOfLetters(String s) {
        return s.length();
   }
}
```

On remarque que cette méthode ressemble au comportement d'un **Function** car elle prend une chaîne en entrée et renvoie un entier. Dans ce cas, il est possible d'utiliser l'opérateur de référence (**::**) pour appeler la méthode **"getNumberOfLetters"** plutôt que de fournir une expression lambda.

```
// Utilisation de la référence à la méthode d'instance d'un objet particulier
StringUtils su = new StringUtils();
Function<String, Integer> function = su::getNumberOfLetters;
```

Référence à une méthode d'instance d'un type particulier



Une méthode de référence vers une méthode d’instance permet d'appeler une méthode existante d’une classe en utilisant l’opérateur de référence (**::**)

Syntaxe : NomDeLaClasse::nomDeLaMethode

Référence à une méthode d'instance d'un type particulier



Exemple:

Dans cet exemple, nous trions une liste de chaînes de caractères à l'aide de la méthode **“compareTo”** de la classe **String**. Nous utilisons la référence de méthode pour faire référence à la méthode **“compareTo”** de chaque chaîne de la liste.

```
List<String> strings = new ArrayList<>();
strings.add("Hello");
strings.add("Welcome");
strings.add("Goodbye");

Collections.sort(strings, String::compareTo);
```

Référence à un constructeur



Une méthode de référence vers un constructeur permet de créer de nouveaux objets en utilisant l'opérateur de référence (**::**)

Syntaxe : NomDeLaClasse::new

Exemple:

Imaginons une classe Point qui définit 2 entiers x et y qui représente les coordonnées d'un point dans un repère orthonormé.

Référence à une méthode d'instance d'un objet particulier

```
public class Point {
    int x;
    int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

On remarque que ce constructeur ressemble au comportement d'un **BiFunction** car il prend 2 entiers en entrée et renvoie un Point . Dans ce cas, il est possible d'utiliser l'opérateur de référence (**::**) pour créer un nouveau objet de type **Point** plutôt que de fournir une expression lambda.

```
// Utilisation de la référence à un constructeur
BiFunction<Integer, Integer, Point> bif = Point::new;

Point p = bif.apply(3, 2);
```

Merci pour votre attention

